Teoría de la Computación

Proyecto 2: Algoritmo CYK y simplificación de gramáticas

Josue Hernández

Juan Daniel Ordóñez

José Sánchez

Octubre de 2026

Introducción

En este proyecto implementamos un programa que decide si una frase en inglés pertenece al lenguaje generado por una gramática libre de contexto. La decisión la toma el algoritmo de Cocke, Younger y Kasami (CYK), que solo funciona cuando la gramática está en Forma Normal de Chomsky (CNF). Por eso el trabajo tiene dos partes: primero simplificamos la gramática hasta dejarla en CNF y después aplicamos CYK sobre ella.

Además del SÍ o NO que pide el enunciado, el programa mide cuánto tarda CYK y construye el parse tree de las frases aceptadas. La gramática de trabajo es la del enunciado, pero el programa lee cualquier gramática desde un archivo de texto, así que también la probamos con una gramática de expresiones aritméticas.

Diseño de la aplicación

Planteamos el programa como una cadena de pasos. Cada paso recibe una gramática y devuelve otra nueva, sin modificar la que recibió:

```
archivo -> Grammar -> sin ϵ -> sin unarias -> sin inútiles -> CNF -> CYK -> árbol
```

Una gramática se guarda como un diccionario que asocia cada variable con la lista de sus cuerpos; cada cuerpo es una tupla de símbolos y la cadena vacía es la tupla vacía. Consideramos variable a todo símbolo que aparece a la izquierda de alguna regla y terminal a todo lo demás, de modo que no hace falta que las variables estén en mayúsculas. Esto nos permitió usar sin cambios símbolos como `id`, `+` o `(` en la gramática aritmética.

Cada paso vive en su propio archivo:

Archivo	**Contenido**
`grammar.py`	clase `Grammar` y lector de archivos
`epsilon.py`, `unit.py`, `useless.py`	los tres pasos de simplificación
`cnf.py`	conversión a Forma Normal de Chomsky
`simplify.py`	une los pasos anteriores (`simplify` y `to_cnf`)
`cyk.py`	algoritmo CYK y construcción del árbol
`main.py`	lectura de la frase, salida y medición del tiempo

El archivo de una gramática tiene una regla por línea, con los símbolos separados por espacios; las alternativas van separadas por `|`, la cadena vacía se escribe `e` y la primera regla define el símbolo inicial. Si una línea está mal formada, el lector avisa en cuál.

Simplificación de la gramática

Producciones vacías. Primero calculamos qué variables pueden derivar la cadena vacía (las anulables): una variable lo es si tiene la regla `A -> ϵ` o si alguno de sus cuerpos está formado solo por variables anulables, y repetimos hasta que no aparezcan más. Luego, para cada regla generamos todas las versiones que omiten algún subconjunto de sus símbolos anulables y descartamos la versión vacía. Si un cuerpo tiene `m` símbolos anulables salen `2^m` versiones; por ejemplo, con `X` anulable, `E -> T X` pasa a `E -> T X | T`. El lenguaje que queda es el original sin la cadena vacía.

Producciones unarias. Una producción unaria es de la forma `A -> B` con `B` variable. Calculamos los pares `(A, B)` tales que `A` llega a `B` usando solo reglas unarias, y para cada par copiamos a `A` las reglas de `B` que no son unarias. Cada par se procesa una sola vez, así que los ciclos no son problema.

Símbolos inútiles. Un símbolo es inútil si no deriva ninguna cadena de terminales o si no se alcanza desde el símbolo inicial. Los quitamos en ese orden: primero las variables que no generan nada, y después lo que ya no se alcanza. El orden importa. En `S -> a | B`, `B -> A b`, `A -> A a`, la variable `B` parece alcanzable, pero no genera nada; si empezáramos por la alcanzabilidad `A` y `B` se quedarían en la gramática.

Forma Normal de Chomsky

Después de simplificar solo faltan dos casos. Cuando un terminal aparece en un cuerpo de dos o más símbolos, lo reemplazamos por una variable nueva `T_n` que produce únicamente ese terminal, y reutilizamos la misma variable cada vez que el terminal vuelve a aparecer. Cuando un cuerpo tiene `k >= 3` símbolos, lo partimos en una cadena de reglas con dos variables usando `k - 2` auxiliares `C_n`. Antes de crear un nombre revisamos todos los símbolos de la gramática, para que `T_1` o `C_1` no choquen con uno que ya exista.

El algoritmo CYK

CYK es un caso de programación dinámica. Para una frase de `n` palabras llenamos una tabla triangular en la que la celda `(i, l)` guarda las variables que generan la subcadena que empieza en la posición `i` y tiene longitud `l`. Las celdas de longitud 1 se llenan con las variables `A` que cumplen `A -> palabra`. Para longitudes de 2 a `n` probamos, en cada subcadena, todos los puntos de corte: si `B` está en la parte izquierda, `C` en la derecha y existe `A -> B C`, entonces `A` pertenece a la celda. La frase se acepta si el símbolo inicial está en la celda que cubre todas las palabras. El costo es `O(n^3 · |P|)` en tiempo y `O(n^2 · |V|)` en espacio.

Para el parse tree no basta con guardar un valor booleano por variable. Cada variable de cada celda guarda cómo se obtuvo (la palabra, o el corte y el par `B C` usados), y al terminar reconstruimos el árbol siguiendo esas referencias desde el símbolo inicial. Cuando hay más de una derivación posible conservamos la primera que se encuentra. Un nodo del árbol es `(símbolo, [hijos])` y una hoja es `(palabra, [])`.

Un ejemplo completo

Partimos de la gramática `grammars/1.txt`:

```
E -> T X
X -> + T X | ϵ
T -> F Y
Y -> * F Y | ϵ
F -> ( E ) | id
```

Las variables anulables son `X` y `Y`, y los pares unarios que aparecen después de quitar las vacías son `(E,T)`, `(T,F)` y `(E,F)`. Con eso la gramática queda así, y no tiene símbolos inútiles:

```
E -> T X | F Y | ( E ) | id
X -> + T X | + T
T -> F Y | ( E ) | id
Y -> * F Y | * F
F -> ( E ) | id
```

En CNF, `T_1` y `T_2` sustituyen a los paréntesis, `T_3` al `+` y `T_4` al `*`, y las `C_n` parten los cuerpos de tres símbolos:

```
E -> T X | F Y | T_1 C_1 | id        T_1 -> (       C_1 -> E T_2
X -> T_3 C_2 | T_3 T                T_2 -> )       C_2 -> T X
T -> F Y | T_1 C_3 | id             T_3 -> +       C_3 -> E T_2
Y -> T_4 C_4 | T_4 F                T_4 -> *       C_4 -> F Y
F -> T_1 C_5 | id                                  C_5 -> E T_2
```

Con esa gramática ejecutamos el programa sobre `( id * id ) + id`. En el árbol se ven las variables auxiliares que introdujo la conversión. Las ramas están dibujadas en ASCII, que es lo que imprime el programa en consolas sin soporte de Unicode:

```
$ python3 main.py -g grammars/1.txt "( id * id ) + id"
Frase: ( id * id ) + id
Resultado: SÍ
Tiempo de CYK: 0.0283 ms

Parse tree:
E
|-- T
|   |-- T_1
|   |   `-- (
|   `-- C_3
|       |-- E
|       |   |-- F
|       |   |   `-- id
|       |   `-- Y
|       |       |-- T_4
|       |       |   `-- *
|       |       `-- F
|       |           `-- id
|       `-- T_2
|           `-- )
`-- X
    |-- T_3
    |   `-- +
    `-- T
        `-- id
```

Pruebas

Con la gramática en inglés del enunciado, que ya viene en CNF, el programa acepta las frases bien formadas y rechaza las que tienen las palabras en mal orden, las incompletas y las que usan palabras fuera del vocabulario:

```
$ python3 main.py "she eats a cake with a fork"
Frase: she eats a cake with a fork
Resultado: SÍ
Tiempo de CYK: 0.0227 ms
...

$ python3 main.py "cake the eats she"
Frase: cake the eats she
Resultado: NO
Tiempo de CYK: 0.0117 ms

$ python3 main.py "she eats a pizza"
Frase: she eats a pizza
Resultado: NO
Palabras fuera de la gramática: pizza
Tiempo de CYK: 0.0124 ms
```

El resto de las frases que probamos:

Frase	Resultado	Tiempo
the cat drinks the beer	SÍ	0.016 ms
he cooks	SÍ	0.010 ms
she cuts the meat in the oven with a knife	SÍ	0.037 ms
the cat	NO	0.009 ms
she eats with	NO	0.010 ms
id + * id (gramática aritmética)	NO	0.012 ms

Los tiempos corresponden a una sola ejecución y cambian de una corrida a otra. Para frases de este tamaño CYK tarda decenas de microsegundos.

Además de las frases, escribimos 89 pruebas automáticas. Cubren el lector de archivos y sus errores, cada paso de simplificación con resultados exactos sobre `1.txt`, la conversión a CNF, CYK (frases aceptadas y rechazadas, orden de las hojas del árbol, gramática ambigua, palabra desconocida, entrada vacía) y la ejecución completa de `main.py`. Una parte importante de las pruebas compara el lenguaje: generan todas las cadenas cortas de la gramática original y de la transformada y verifican que sean las mismas, salvo la cadena vacía.

Discusión

El orden de los pasos de simplificación no es arbitrario. Eliminamos las producciones vacías antes que las unarias porque quitar una vacía puede crear reglas unarias nuevas (`E -> T X` con `X` anulable produce `E -> T`), y dejamos los símbolos inútiles para el final porque los pasos anteriores pueden dejar variables sin uso.

Al eliminar las producciones vacías la cadena vacía deja de pertenecer al lenguaje, y CYK sobre una gramática en CNF tampoco la reconoce, por lo que una frase vacía siempre se rechaza. También notamos que la simplificación puede hacer crecer la gramática (`2^m` versiones por regla y variables auxiliares en la CNF), aunque con las gramáticas del proyecto el tamaño siguió siendo pequeño.

Usamos `e` para la cadena vacía porque así viene en `1.txt`; la consecuencia es que un terminal llamado literalmente `e` solo puede usarse junto con otros símbolos. Otra limitación es que si una gramática es ambigua el programa muestra solo una de las derivaciones posibles.

Un obstáculo que encontramos al integrar las partes fue la versión de Python: la primera versión de `cyk.py` usaba `typing.TypeAlias`, que existe desde la 3.10, y no se podía importar en la 3.9 instalada en uno de nuestros equipos. Lo resolvimos con tipos de `typing` compatibles y mejoramos el mensaje de error de `main.py`, que en ese momento decía que faltaba el archivo cuando en realidad fallaba al importarse. Como recomendación, conviene probar la

simplificación por separado antes de usar CYK: comparar las cadenas que genera la gramática antes y después de transformarla permite detectar errores de conversión que, de otro modo, se confundirían con fallos del algoritmo.

Cómo ejecutarlo

El programa requiere Python 3.9 o superior y no usa librerías externas.

```
python3 main.py "She eats a cake with a fork"
python3 main.py --steps "the cat drinks the beer"
python3 main.py -g grammars/1.txt "( id * id ) + id"
python3 -m unittest discover -s tests -t .
```

Con `--steps` se imprimen la gramática original, la simplificada y la CNF antes de validar la frase. Si no se indica la frase, el programa la pide por consola. El último comando ejecuta las pruebas automáticas.